Relazione Progetto SDCC - Traccia B4

Pepe Federico (Matricola: 0365363)*†
*Dipartimento di Ingegneria, Università degli Studi di Roma "Tor Vergata", Roma, Italia
federico.pepe@students.uniroma2.eu

***Abstract*— Questo progetto presenta un'architettura per il monitoraggio e l'automazione industriale ispirata allo standard WirelessHART. Il sistema gestisce la raccolta e l'elaborazione di flussi continui di dati generati da un cluster di sensori IoT. Per ovviare all'impossibilità di una trasmissione diretta verso l'applicazione utente, l'infrastruttura introduce un elemento intermedio con funzioni di instradamento, gestione dei dati e attuazione dei comandi, ottimizzando l'affidabilità e la scalabilità della rete.**

***Index Terms*—IoT Industriale, WirelessHART, Middleware, Monitoraggio, Automazione di Fabbrica.**

I. Introduction

Negli ultimi anni, l'evoluzione dell'Industrial Internet of Things (IIoT) e l'affermazione del paradigma dell'Industria 4.0 hanno trasformato radicalmente i processi produttivi, rendendo l'acquisizione in tempo reale dei dati di campo un requisito fondamentale per il monitoraggio e il controllo dei sistemi industriali. In questo scenario, le reti di sensori wireless (*Wireless Sensor Networks*, WSN) svolgono un ruolo cruciale, offrendo un'elevata flessibilità operativa e riducendo drasticamente i costi di cablaggio all'interno degli impianti.

A. Motivazione e Obiettivi

L'obiettivo di questo lavoro consiste nella progettazione e nell'implementazione di un'architettura per il monitoraggio e l'automazione di fabbrica che trae ispirazione dalle specifiche di affidabilità e gestione del traffico dello standard WirelessHART.

Il sistema si basa su un cluster di nodi sensoriali IoT preposti al campionamento e alla generazione continua di flussi dati. Tali flussi vengono instradati ed inviati a un'applicazione di monitoraggio centrale, la quale si articola in tre fasi principali:

- **Ingestion dei dati:** fase dedicata al pre-processamento e alla validazione del flusso informativo, orientata allo scarto di valori anomali, fuori scala o nulli;
- **Analisi in tempo reale:** elaborazione e sintesi dei dati attraverso logiche di *Stream Processing*;
- **Processo decisionale e attuazione:** valutazione dello stato operativo dei dispositivi sulla base dei dati elaborati, con eventuale invio di comandi di arresto preventivo.

II. Architettura del Sistema

L'architettura del sistema si articola in molteplici microservizi e applicativi di supporto. Di seguito viene illustrata la composizione dell'infrastruttura, unitamente alle scelte architetturali adottate per ciascun componente.

A. Framework Utilizzati

- **Apache Kafka [1]:** Opera come *event bus* distribuito e dorsale di messaggistica asincrona per i dati di telemetria. Sebbene gRPC sia stato impiegato per le comunicazioni dirette e a bassa latenza verso il `sink-service` e il `front-end`, il suo utilizzo per lo streaming continuo avrebbe introdotto un forte accoppiamento e vulnerabilità ai picchi di carico. L'adozione di Kafka garantisce:
  - **Disaccoppiamento e Backpressure:** Separazione temporale e logica tra i servizi, consentendo a ciascun componente di elaborare i flussi al proprio ritmo ed assorbire i picchi di traffico.
  - **Resilienza e Replayability:** La persistenza su *commit log* e la gestione dei checkpoint nell'`analyzer-service` prevengono la perdita di dati e permettono il ripristino dello stato in caso di crash dei nodi, scenario in cui una connessione gRPC pura fallirebbe.
  - **Elaborazione Distribuita:** Trasferimento della logica di calcolo sui microservizi in Go, demandando a Kafka la gestione dello stato, della tolleranza ai guasti e dell'instradamento dei messaggi.
- **Redis [2]:** Impiegato come datastore *in-memory* per la consultazione a bassissima latenza dei parametri operativi dei sensori da parte del `Decision-Service`. Svolge inoltre il ruolo di motore di notifica in tempo reale verso la UI tramite la funzionalità integrata di Pub/Sub su canali dedicati. L'uso di Redis Pub/Sub al posto di Kafka per l'interfaccia utente è stato motivato da:
  - **Notifiche Effimere:** Gli aggiornamenti della dashboard sono dati di stato temporanei che beneficiano dell'instradamento esclusivamente in memoria di Redis, evitando l'overhead d'I/O su disco del *commit log* di Kafka.
  - **Semplicità e Broadcast:** Eliminazione della gestione di *offset* e *Consumer Groups*, sproporzionati per messaggi in *broadcast* verso client web connessi che non richiedono lo storico degli eventi in caso di disconnessione.
- **InfluxDB [3]:** Adottato come *Time-Series Database* (TSDB) per la gestione nativa e ad alte prestazioni dei dati di telemetria temporizzati. InfluxDB risponde a due esigenze primarie della pipeline:
  - **Storicizzazione ad Alto Throughput (`write-service`):** Permette l'ingestione continua e a bassa latenza dei dati grezzi provenienti

dall'`ingestion-service`, sfruttando motori di scrittura ottimizzati per operazioni *append-only*.

  - **Persistenza e Consultazione Storica (`sink-service`):** Memorizza le metriche aggregate ed elaborate dall'`analyzer-service`, garantendo query analitiche veloci per la dashboard.

- **Prometheus & Grafana**: Prometheus raccoglie periodicamente le metriche esposte dal sistema, mentre Grafana funge da *Data-Source* per visualizzare l'andamento temporale delle metriche, attraverso dashboard interattive.

B. Definizione dei Micro-servizi

- **Generatore di Eventi:** è il modulo responsabile della definizione e della gestione del cluster di sensori IoT. Al fine di simulare uno scenario multi-applicazione con dispositivi eterogenei, sono state modellate due specifiche tipologie di sensore: *TemperatureSensor* e *PressureSensor*. Per ciascun dispositivo vengono definiti i parametri e le soglie limite che guidano il modulo decisionale nell'individuazione della migliore azione operativa sulla base della telemetria ricevuta. Il modulo integra inoltre la logica per la simulazione del comportamento operativo dei nodi, modellando la generazione dei dati secondo le seguenti percentuali:
  - **1% dei casi:** dati corrotti o malformati;
  - **2% dei casi:** dati fuori scala (*out-of-bounds*);
  - **97% dei casi:** dati validi ed allineati ai parametri operativi.

  Infine, il componente gestisce il ciclo di vita dei messaggi di feedback: al ricevimento di un comando di arresto da parte del sistema centrale, simula l'interruzione preventiva del sensore, la successiva fase di ripristino/riparazione e il reinserimento in servizio del dispositivo.

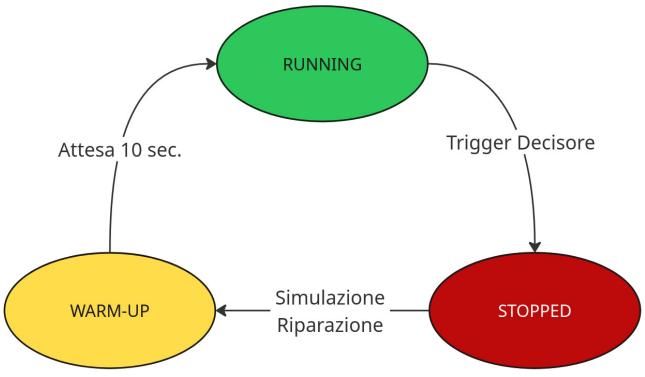


Fig. 1. Architettura generale dello stato dei sensori IoT.

- **Ingestion-Service:** Responsabile dell'acquisizione e del pre-processamento primario dei dati dal cluster di sensori. Esegue il *data cleaning* (filtraggio di misurazioni corrotte, nulle o fuori scala) e la normalizzazione delle strutture dati, garantendo la pulizia e la formattazione uniforme del payload prima dell'invio ai moduli di analisi.
- **Analyzer-Service:** rappresenta il cuore computazionale della pipeline di streaming, responsabile dell'elaborazione analitica dei dati sensoriali. Il servizio è progettato secondo un'architettura altamente concorrente e basata su eventi, strutturata in due livelli principali: un *Dispatcher* centrale e una serie di *Worker* concorrenti allocati dinamicamente per ogni singolo sensore.
  - **Gestione della Concorrenza (Worker per Sensore):** Durante la lettura continua dei messaggi dai topic Kafka, il *main loop* deserializza gli eventi e instrada i dati in ingresso tramite una mappa concorrente sincrona (`sync.Map`). Se viene rilevato un nuovo sensore, il sistema avvia una goroutine dedicata (`SensorWorker`) associata a un canale Go con buffer (`InputChannel`). Questo approccio garantisce l'elaborazione isolata e in ordine sequenziale degli eventi per ciascun sensore.
  - **Gestione degli Eventi di Controllo e Invalidation:** Il servizio integra una sottoscrizione al canale Pub/Sub di Redis (`sensor:state-events`). In presenza di eventi di cambio stato (es. la transizione di un sensore allo stato `STOPPED`), il *Dispatcher* notifica il `SensorWorker` competente tramite un canale dedicato di invalidazione (`InvalidDataChannel`), consentendo di sospendere l'elaborazione o pulire tempestivamente lo stato locale della finestra analitica.
  - **Tolleranza ai Guasti, Snapshotting e Commit Atomico:** Per garantire la continuità operativa senza perdita di stato in caso di guasto, il servizio implementa un meccanismo di *checkpointing* periodico (eseguito ogni 30 secondi o su richiesta di invalidazione). Il processo avviene tramite una procedura a due fasi in coordinamento con Kafka e Redis:
    1) Viene acquisito uno *snapshot* consistente in memoria RAM dello stato delle finestre analitiche e persistito su Redis (`SaveSnapshot`).
    2) Solo a seguito del positivo salvataggio su Redis, il servizio esegue il commit atomico degli *offset* dei messaggi su Kafka per le singole partizioni consumate.

    In fase di avvio (*Recovery Phase*), il servizio carica da Redis gli ultimi stati salvati (`LoadSnapshot`), consentendo ai nuovi `SensorWorker` di riprendere l'analisi esattamente dal punto dell'ultimo checkpoint garantendo la consistenza del dato.

1) Gestione dello Stato e Finestre Analitiche: Il WorkerInstance: Ogni sensore attivo nell'architettura viene gestito in maniera indipendente da una goroutine dedicata, istanziata mediante la struttura `WorkerInstance`. Questo componente incapsula lo stato temporale e il buffer dei dati del sensore, eseguendo i calcoli statistici

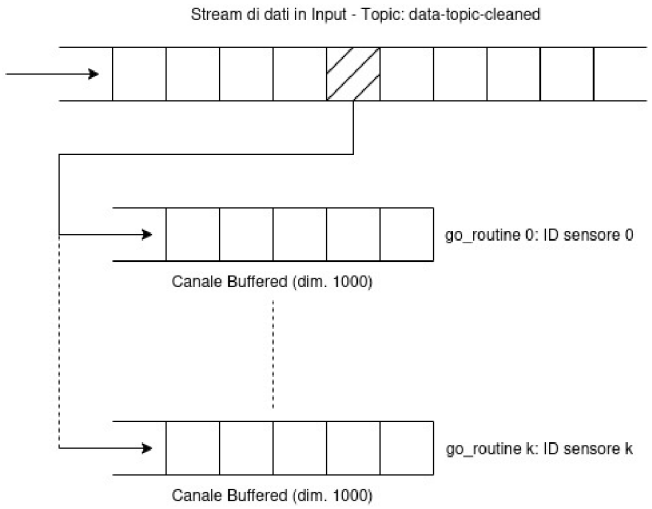


Fig. 2. Dispatcher messaggi.

in isolamento concorrente tramite costrutti di sincronizzazione RWMutex (`sync.RWMutex`).

- **Gestione della Finestra Scorrevole (Sliding Window):** L'analisi dei dati si basa su una *Sliding Window* temporale definita da una dimensione di osservazione di 60 secondi (`windowDuration`) e un intervallo di scorrimento/valutazione di 30 secondi (`slideInterval`). A ogni nuovo evento valido ricevuto, il dato viene inserito nel buffer locale e gli elementi antecedenti il limite di inizio finestra (`windowStart = MaxEventTime - 60s`) vengono progressivamente eliminati tramite operazione di *slicing*, mantenendo ridotta l'impronta di memoria.
- **Gestione dei Dati Tardivi tramite Watermark:** Per gestire il disallineamento temporale causato da latenze di rete o disconnessioni dei dispositivi, il worker implementa il concetto di *Watermark* basato su *Event-Time*. Il watermark è calcolato dinamicamente come:

$$\text{Watermark} = \text{MaxEventTime} - \text{watermarkDelay}$$
$$(\text{con watermarkDelay} = 15\text{s})$$

  Tutti i messaggi il cui marcatore temporale risulta antecedente al valore corrente di watermark vengono considerati "late data" non recuperabili ed esplicitamente scartati prima dell'inserimento nel buffer, garantendo la correttezza dei calcoli statistici.
- **Algoritmi Analitici e Calcolo Statistico (`processWindow`):** Allo scadere di ogni intervallo di slide (30s), una copia thread-safe del buffer viene passata alla funzione `processWindow`. Per evitare instabilità numeriche dovute all'accumulo di valori in virgola mobile, la media e la deviazione standard vengono calcolate in un unico passaggio tramite l'**Algoritmo di Welford**. Contestualmente, viene stimato il tasso di variazione temporale (*Rate of Change*) applicando una *regressione lineare* sui minimi quadrati rispetto al tempo trascorso dal primo campionamento. I risultati aggregati vengono marcati con un id unico `ULID` e pubblicati sul topic Kafka `processed-data-topic`.
- **Checkpointing e Rilevamento dello Stato (`CaptureWindowSnapshot`):** La funzione `CaptureWindowSnapshot` consente l'ispezione thread-safe dello stato interno di tutti i worker attivi. Tramite blocco in sola lettura (`RLock`), viene creata una copia sia delle variabili temporali (`MaxEventTime`, `Watermark`, `LastEvaluation`) sia delle fette di buffer dei singoli sensori. Questa fotografia dello stato viene poi serializzata e salvata su Redis durante la procedura globale di checkpoint.
- **Invalidazione dei Messaggi e Gestione delle Transizioni di Stato:** Il worker ascolta costantemente il canale di controllo `invalidDataChannel` per reagire asincronamente alle istruzioni inviate dal sistema o dalla dashboard:
  - **Stato `STOPPED` (`handlerInvalidData`):** Quando un sensore viene arrestato, il worker recupera da Redis il timestamp preciso dell'evento di stop (`stopTime`). Per prevenire la processazione di dati in coda o in ritardo generati dopo l'arresto, il worker imposta la flag `isStopped = true` e filtra il buffer residuo, scartando tutti gli elementi aventi $t >$ stopTime. Successivamente invoca un checkpoint immediato.
  - **Stato `WARM-UP` (`handlerWarmUpState`):** In fase di riavvio o ripristino di un sensore, il worker azzera completamente il buffer residuo della sessione precedente e resetta a zero tutti i contatori temporali (`MaxEventTime`, `Watermark`, `LastEvaluation`), garantendo che la nuova finestra parta da uno stato pulito senza inquinare le serie storiche.
- **Decision-Service:** Modulo di monitoraggio e reattività in tempo reale che consuma i dati aggregati da Kafka. Per ogni evento di tipo `DATA`, recupera i parametri del sensore da Redis (`LoadParameterSensor`) e valuta le anomalie tramite tre vincoli (`evaluateAnomalies`):
  1) *Soglie Operative:* Valore medio $\mu \in [S_{\min}, S_{\max}]$.
  2) *Instabilità:* Deviazione standard $\sigma \leq \sigma_{\max}$.
  3) *Deriva Temporale:* Tasso di variazione $|\text{RateOfChange}| \leq D_{\max}$.

  In caso di violazione, invia un allarme (`ModeStop`) su Kafka per arrestare il sensore ed esegue il commit atomico dell'offset.

C. Separazione delle Responsabilità: Pattern CQRS

La persistenza è organizzata su due percorsi distinti, uno per ciascun livello di astrazione del dato, entrambi alimentati da Kafka:

- **Write-Service:** consuma `data-topic-cleaned` e persiste in modo asincrono su InfluxDB (`WritePoints`) le letture di telemetria grezze (*pressione* e *temperatura*), costituendo lo storico puntuale e isolando l'ingestione ad alto throughput dalle operazioni di lettura.
- **Sink-Service:** consuma `processed-data-topic`, che contiene gli aggregati di finestra prodotti dallo stream processing, ed è il componente in cui il pattern **CQRS** (*Command Query Responsibility Segregation*) è applicato in senso proprio.

Seguendo l'indicazione di Fowler [4] di circoscrivere il CQRS a un singolo *bounded context*, la separazione è interna a `sink-service`: un percorso di *update*, guidato dagli eventi Kafka, aggiorna in batch il *read model* su InfluxDB, mentre un percorso di *query*, guidato dalle richieste, lo espone tramite l'interfaccia gRPC `GetMetricsSnapshot`. La lettura è semplificata dal calcolo anticipato degli aggregati in `analyzer-service`. Il lato *command* è volutamente sottile: proietta eventi già prodotti a monte, senza logica di dominio. Il vantaggio è poter ottimizzare in modo indipendente due profili di accesso molto asimmetrici, scritture continue e ad alto volume gestite in batch contro letture sporadiche su aggregati già pronti, senza che le query siano vincolate al ritmo di consumo da Kafka.

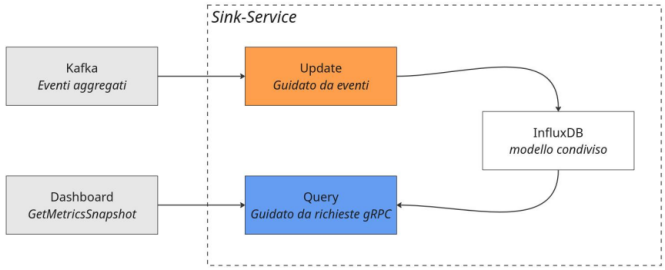


Fig. 3. Pattern CQRS, visione del Sink-Service.

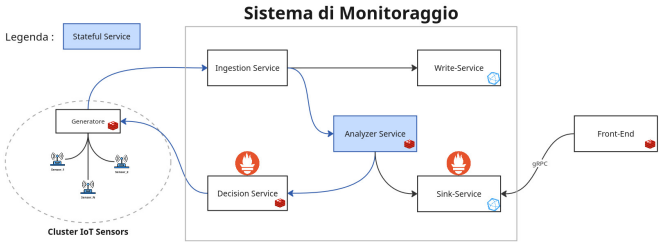


Fig. 4. Visione ad alto livello del Sistema.

III. Deployment su Cloud

L'architettura event-driven è stata orchestrata e distribuita su Amazon Web Services (AWS) tramite istanze EC2.

A. Engine di Orchestrazione: K3s

Si è adottato K3s, una distribuzione Kubernetes lightweight che consolida il Control Plane in un singolo binario e sostituisce `etcd` con SQLite. Questo minimizza il footprint di risorse sui nodi, massimizzando la memoria e la CPU allocabili ai microservizi e ai broker.

B. Topologia del Cluster e Separazione dei Ruoli

Il cluster comprende 5 istanze EC2 (1 Control Plane e 4 Worker) isolate per dominio funzionale:

- **Control Plane (Master):** Dedicato esclusivamente all'API Server, allo scheduler e al controller manager, senza workload applicativi.
- **Worker 1 (load):** Dedicato all'`events-generator` per la gestione del traffico generato dai sensori.
- **Worker 2 (core):** Riservato all'elaborazione CPU-bound dei servizi.
- **Worker 3 (data):** Riservato ai database temporanei/storici ed ai servizi di scrittura (`sink-service` e `write-service`).
- **Worker 4 (kafka):** Ospita Apache Kafka ed il relativo exporter, isolando l'I/O di rete della coda.

C. Scalabilità Orizzontale Automatica (HPA)

L'autoscaling orizzontale impiega una politica ibrida basata sia su metriche di risorsa che su *Custom Metrics* applicative, con un range di 1–6 pod per microservizio.

1) Soglie e Metriche Target:

- **CPU e Memoria:** Target fissati al 70% della CPU e all'80% della memoria richieste.
- **Kafka Consumer Lag:** `Analyzer-Service` e `Decision-Service` valutano la metrica `kafka_consumergroup_lag_by_group` (esposta via `prometheus-adapter`). Lo scaling scatta al superamento di un *AverageValue* per pod pari a 5.000 messaggi per l'`Analyzer` e 1.000 per il `Decision`.

2) Stabilizzazione: Per evitare fenomeni di *flapping* e garantire reattività ai picchi:

- **Scale-Up (`stabilizationWindowSeconds: 0`):** Espansione immediata tramite `selectPolicy: Max`, che raddoppia le repliche (+100% ogni 15 secondi) al superamento delle soglie.
- **Scale-Down (`stabilizationWindowSeconds: 180`):** Raffreddamento di 3 minuti con contrazione graduale del −25% ogni 60 secondi.

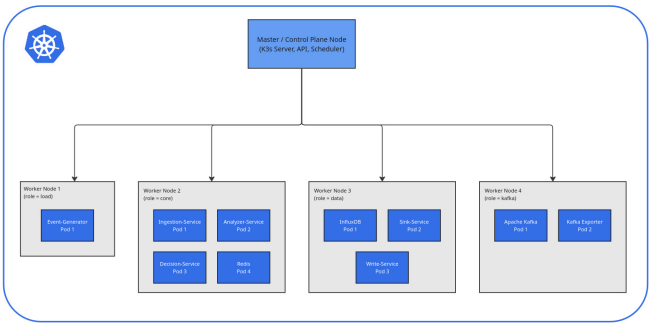


Fig. 5. Cluster K3s.

IV. Fase Sperimentale e Metodologia di Validazione

Per valutare la scalabilità e la reattività dell'architettura a microservizi sotto diversi regimi di carico, è stata predisposta un'infrastruttura di monitoraggio basata su metriche personalizzate, raccolte ed aggregate tramite *Prometheus*.

A. Metriche di Prestazione Applicative

L'analisi delle prestazioni del sistema si concentra su tre indicatori chiave:

- **Throughput (eventi/s):** Derivato dal contatore vettoriale `ProcessedEventsTotal`, incrementato ad ogni evento elaborato con successo o terminato in errore. Il throughput effettivo viene calcolato applicando la funzione `rate()` di Prometheus su una finestra temporale definita.
- **Processing Lag (Backpressure):** Misura il tempo di stazionamento del messaggio in coda, determinato come la differenza tra l'istante di lettura da parte del consumer e il timestamp nativo impresso dal broker Kafka all'atto della pubblicazione.
- **Latenza End-to-End:** Rilevata tramite l'iniezione periodica (ogni 500 ms) di messaggi sintetici di controllo (`LATENCY_MARKER`). La latenza viene calcolata come differenza tra l'istante di completamento dell'elaborazione e il timestamp di ingresso nell'architettura:

$$L_{\text{E2E}} = t_{\text{processing}} - t_{\text{ingress_nano}} \tag{1}$$

Per l'analisi delle prestazioni si è adottata la media aggregata sulla finestra di un minuto (`rate(sum)/rate(count)`), anziché un percentile: data la natura del fenomeno di backpressure osservato — condiviso in modo pressoché uniforme da tutti i messaggi in transito sul broker saturo, piuttosto che concentrato su pochi outlier isolati — la media risulta comunque rappresentativa dell'andamento generale della latenza sotto carico.

B. Configurazione della Campagna di Test

L'infrastruttura del cluster K3s è stata distribuita su istanze Amazon EC2 di tipo *t3.medium*, che offrono un compromesso bilanciato tra capacità computazionale e sostenibilità dei costi operativi.

Al fine di individuare il vero limite operativo del sistema, si è adottata volutamente una configurazione di Apache Kafka basata su un singolo broker. Sebbene un unico broker costituisca il primario collo di bottiglia architetturale sia per l'I/O di rete che per il disco, tale scelta permette di:

1) Isolare la capacità massima di smaltimento del bus di messaggistica;
2) Valutare l'efficacia delle politiche di autoscaling orizzontale (HPA) nel compensare la saturazione progressiva delle code;
3) Monitorare la stabilità dell'architettura al raggiungimento della saturazione delle risorse di sistema (CPU, memoria e consumer lag).

I paragrafi seguenti illustrano nel dettaglio i risultati ottenuti nelle diverse condizioni di stress test applicate al cluster.

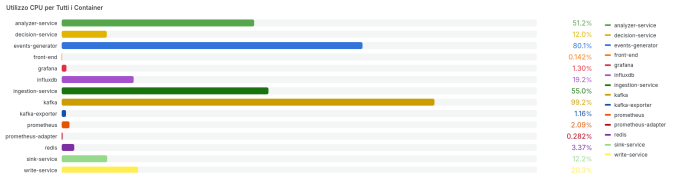


Fig. 6. Utilizzo CPU per Tutti i Container.

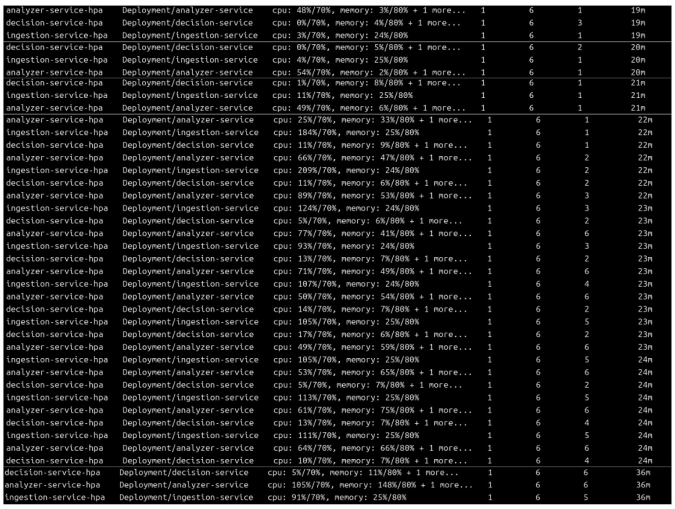


Fig. 7. Reazione dell'Autoscaling (HPA).



Fig. 8. Kafka Network Ingest Rate.

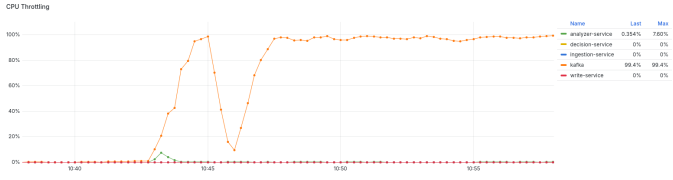


Fig. 9. CPU Throttling.

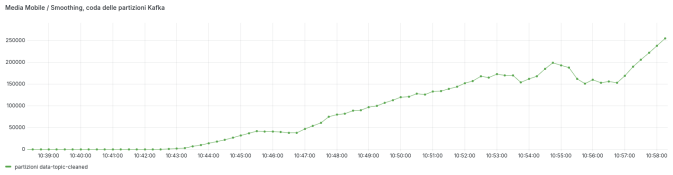


Fig. 10. Accodamento data-topic-cleaned.

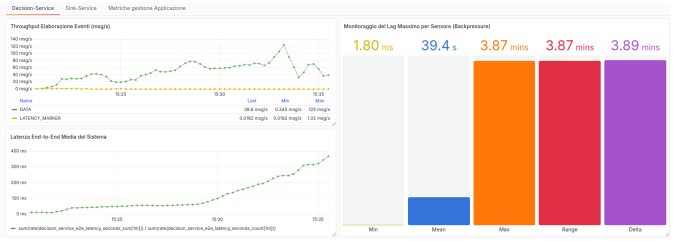


Fig. 11. Metriche Decision-service.

C. Analisi dei Risultati

Per validare la scalabilità orizzontale del sistema, è stato condotto un test di carico a rampa fino a 12 000 sensori attivi, analizzando la reattività dell'*autoscaling* e i limiti dell'architettura. I risultati confermano la corretta attivazione dell'autoscaling per `ingestion-service`, `analyzer-service` e `decision-service`, tutti giunti al massimo di 6 repliche, tranne `ingestion-service` fermo a 5: dato l'accumulo ininterrotto di lag osservato su `data-topic-cleaned` (oltre 250 000 eventi, ben oltre le soglie configurate), è verosimile che la metrica esterna di lag — e non la sola pressione di CPU — abbia costituito il principale driver dello scaling verso il massimo consentito. `ingestion-service` ha inoltre registrato picchi di CPU del 209% rispetto al target del 70%, segno che la rapida crescita del carico supera comunque i tempi di reazione dell'HPA anche con *scale-up* aggressivo. Ogni evento di scale-out ha infine causato cali temporanei di throughput su `decision-service`, dovuti al fisiologico *rebalance* dei consumer group Kafka.

Il principale collo di bottiglia è risultato il broker Kafka, esclusivamente a livello di CPU: pur isolato su nodo dedicato, ha raggiunto quasi completa saturazione (99.2% di utilizzo, *throttling* sostenuto al 99.4% per la seconda metà del test), a fronte di un traffico di rete contenuto (picco di soli 7.39 Mb/s, ben lontano da qualunque limite di banda) — a conferma che il collo di bottiglia è di natura computazionale e non di I/O di rete. Tale saturazione ha impedito ai servizi a valle di smaltire i messaggi nonostante il numero massimo di repliche attive, causando l'accumulo di lag osservato.

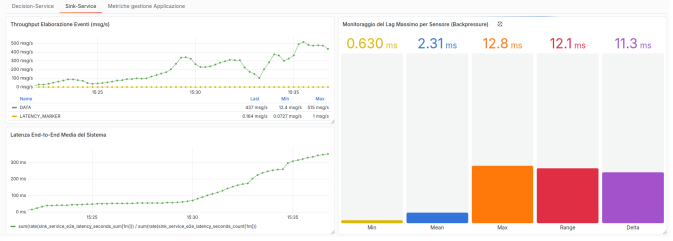


Fig. 12. Metriche Sink-service.

L'impatto della saturazione si è tuttavia manifestato in modo molto diverso tra i due consumer finali. `decision-service` ha sostenuto un throughput medio di 38.6 msg/s (picco 125 msg/s), accumulando un lag medio per sensore di 39.4 s (massimo 3.87 min) e una latenza *end-to-end* di $\sim$ 365 ms. `sink-service`, a parità di broker saturo e stesso topic di origine, ha sostenuto un throughput quasi quattro volte superiore (437 msg/s a regime, picco 515 msg/s) con un lag medio di soli 2.31 ms (massimo 12.8 ms): la sua latenza *end-to-end* ($\sim$ 350 ms), pur simile a quella di `decision-service`, non deriva da elaborazione interna ma dal tempo di permanenza dei messaggi nelle code Kafka a monte. L'asimmetria riflette il diverso costo computazionale per messaggio: `decision-service` esegue valutazioni di anomalia per sensore, mentre `sink-service` si limita a serializzazione e scrittura asincrona su InfluxDB. La saturazione del broker agisce quindi da moltiplicatore, penalizzando in modo sproporzionato il consumer con elaborazione più onerosa per messaggio, a parità di infrastruttura e di flusso dati in ingresso.

V. Conclusioni e Sviluppi Futuri

La campagna di test ha dimostrato l'efficacia dell'architettura a microservizi dal punto di vista dell'elasticità applicativa. Gli Horizontal Pod Autoscaler, configurati con metriche concorrenti di CPU, memoria e lag dei consumer group Kafka, hanno portato `ingestion-service`, `analyzer-service` e `decision-service` quasi al numero massimo di repliche previste (6 pod ciascuno): l'inclusione del lag come metrica esterna si è rivelata una scelta progettuale efficace, permettendo allo scaling di reagire non solo alla pressione di risorse, ma anche all'accumulo di backlog, condizione che si è effettivamente verificata nel test. Nonostante ciò, `ingestion-service` ha toccato picchi di CPU del 209% rispetto al target del 70%, segno che la rapida crescita del carico può superare anche una politica di *scale-up* aggressiva. È stato inoltre osservato, ad ogni evento di scale-out, un calo temporaneo del throughput di `decision-service`, dovuto al *rebalance* dei consumer group Kafka innescato dall'ingresso di nuove repliche: un costo intrinseco di qualunque scaling di consumer basato su Kafka.

Il collo di bottiglia del sistema è risultato il broker Kafka, limitato esclusivamente a livello di CPU: pur isolato su nodo dedicato, ha raggiunto saturazione quasi completa (99.2% di utilizzo, *throttling* al 99.4%), mentre il traffico di rete è rimasto contenuto (picco di 7.39 Mb/s), escludendo la banda come fattore limitante. Tale saturazione ha impedito ai consumer a valle, pur scalati al massimo, di smaltire il flusso in ingresso. L'impatto si è manifestato in modo asimmetrico tra i due consumer finali: `decision-service`, che esegue

valutazioni di anomalia computazionalmente più onerose per messaggio, ha accumulato un lag massimo di 3.87 minuti, contro i soli 12.8 millisecondi di `sink-service` — a dimostrazione di come la saturazione del broker penalizzi in modo sproporzionato i consumer con elaborazione più pesante, a parità di infrastruttura.

A. Sviluppi Futuri

- **Cluster Kafka Multi-Nodo:** distribuzione del broker su più nodi per bilanciare il carico di CPU, principale limite osservato.
- **Tuning Risorse Kafka:** verifica e correzione dei *limits* di CPU assegnati al broker, per garantire margine reale al nodo ospitante senza vincoli incoerenti con la capacità fisica disponibile.
- **Più Partizioni e Consumer:** aumento delle partizioni sui topic per velocizzare lo smaltimento del lag.
- **Scaling Differenziato per Costo Computazionale:** data l'asimmetria osservata tra `decision-service` e `sink-service`, valutare soglie di lag specifiche e più basse per i consumer con elaborazione più onerosa per messaggio, anticipando lo scale-out prima dell'accumulo di backlog significativo.

REFERENCES

[1] Apache Kafka, https://kafka.apache.org.
[2] Redis, https://redis.io/it.
[3] InfluxDB, https://www.influxdata.com.
[4] M. Fowler, "CQRS," https://martinfowler.com/bliki/CQRS.html, Jul. 2011.